

Experiment 2

Client–Server Communication Between Two Virtual Machines

Aim of the Experiment

- Set up two Ubuntu Virtual Machines inside VirtualBox.
- Place both VMs on a shared virtual network.
- Assign and verify IP addresses for each machine.
- Check reachability between the machines using ping.
- Configure one VM as the Server and the other as the Client.
- Write a TCP server script on the Server VM.
- Write a TCP client script on the Client VM.
- Open a TCP connection between the two machines.
- Exchange messages back and forth through a simple chat program.
- Show the chat application working end-to-end.

Background Theory

TCP (Transmission Control Protocol) sets up a reliable, connection-oriented channel between two hosts. In a typical socket-based application, a server binds itself to an IP address and a port number, listens for incoming connection requests, and accepts a client once it connects. After the handshake is complete, both ends can exchange data freely using `send()` and `recv()` calls, and the session ends when either side closes the socket. For this experiment, two Ubuntu virtual machines hosted in VirtualBox were connected through the same internal network and given fixed IP addresses; Python's socket module was then used to build a small two-way chat program running between them.

Setting Up the Network and Assigning IPs

The two virtual machines — "Ubuntu Lab 1" (used as the Server) and "Client" — were connected to the same VirtualBox internal/host-only network. On each machine, the command **ip addr** was first used to inspect the available network interfaces. A fixed IP address was then bound to the `enp0s3` interface of each VM using:

```
sudo ip addr add <ip>/24 dev enp0s3
```

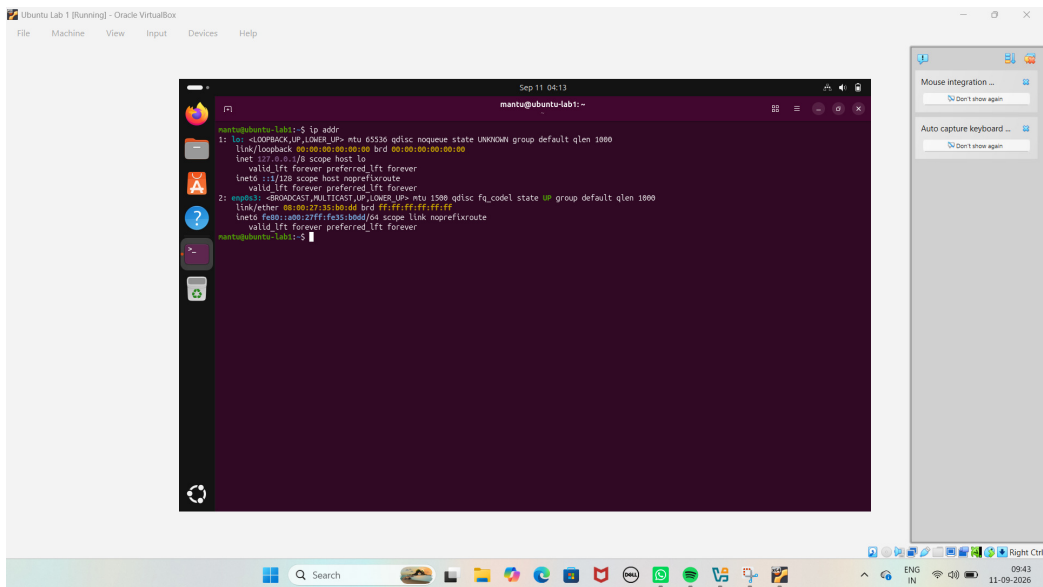


Fig. 1 — Checking the Server VM's network interfaces with `ip addr`, prior to assigning a static IP.

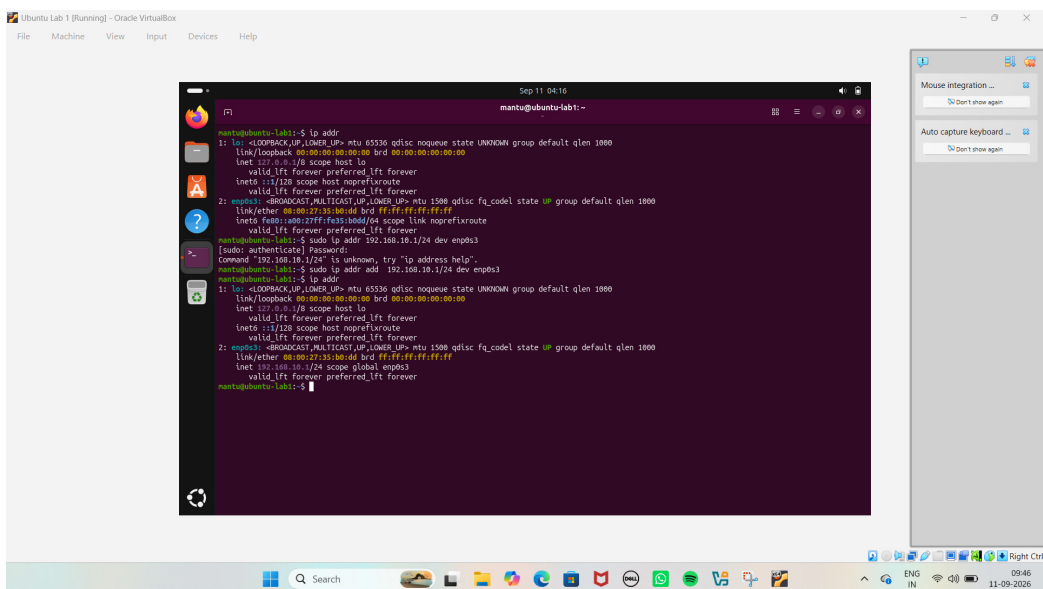


Fig. 2 — Server VM's `enp0s3` interface after assigning it the static address `192.168.10.1/24`.

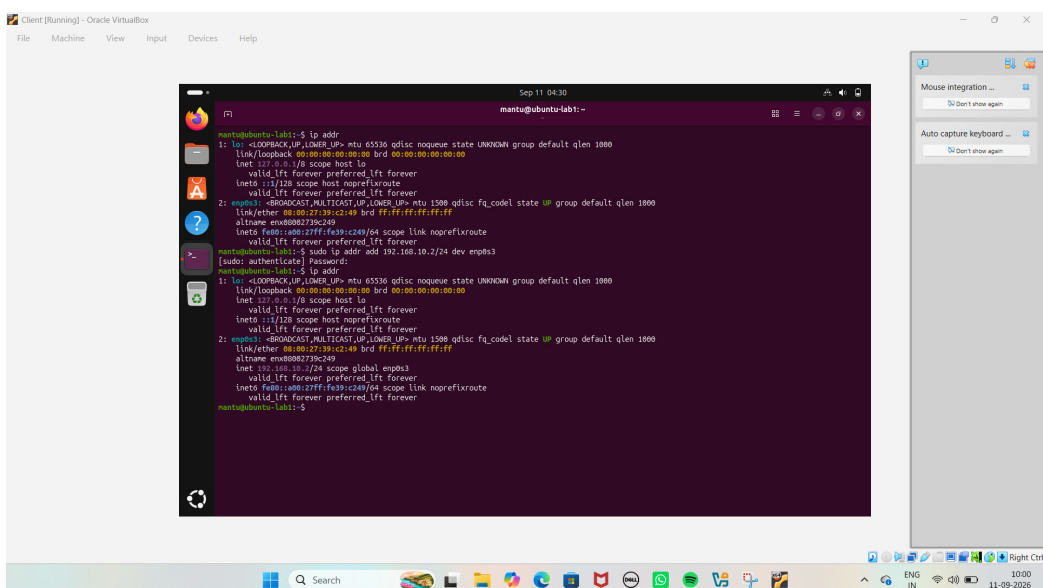


Fig. 3 — Client VM's `enp0s3` interface configured with the static address `192.168.10.2/24`.

Once both interfaces were set up, reachability between the two machines was checked by pinging each VM's address from the other before moving on to the chat program. The successful replies confirmed that the two machines could talk to each other over the virtual network.

Server-Side Script (`server.py`)

This script runs on the machine holding the address `192.168.10.1`. It opens a TCP socket, binds it to the host's IP and port `5000`, waits for a connection, and then repeatedly receives a message from the client and sends back a reply, continuing until either party types "exit".

```
import socket

HOST = "192.168.10.1"
PORT = 5000

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)

print(f"Server started on {HOST}:{PORT}")
print("Waiting for client...")

conn, addr = server_socket.accept()
print(f"Connected by {addr}")

while True:
    message = conn.recv(1024).decode()
    if not message or message.lower() == "exit":
        print("Client disconnected.")
        break
    print("Client:", message)
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break

conn.close()
server_socket.close()
```

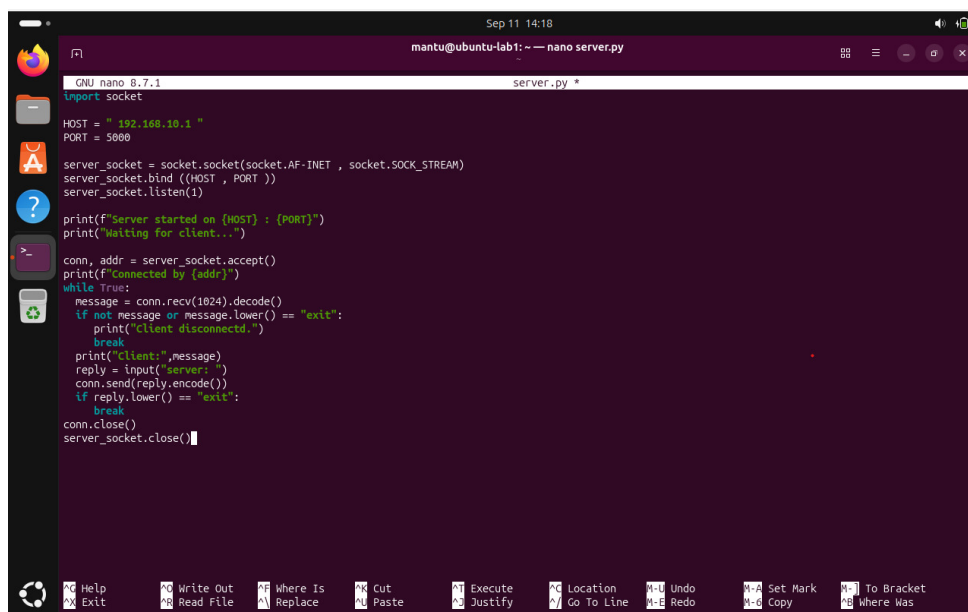
A screenshot of a terminal window titled "mantu@ubuntu-lab1: ~ - nano server.py". The terminal shows the execution of the `server.py` script. The output includes: "Server started on 192.168.10.1:5000", "Waiting for client...", "Connected by ('192.168.10.2', 54321)", "Client: ", "Server: ", "Client disconnected.", and "Server: ". The terminal also shows the source code of `server.py` being edited in nano. The bottom of the terminal displays a list of nano editor shortcuts such as "Help", "Write Out", "Where Is", "Cut", "Execute", "Location", "Undo", "Set Mark", "To Bracket", "Exit", "Read File", "Replace", "Paste", "Justify", "Go To Line", "Redo", "Copy", and "Where Was".

Fig. 4 — `server.py` open in the nano editor on the Server VM.

Client-Side Script (`client.py`)

This script runs on the second machine (192.168.10.2). It opens a TCP socket, connects to the server's IP and port, and then loops through sending a message and waiting for the server's reply, again ending when either side sends "exit".

```
import socket

server_ip = "192.168.10.1"
port = 5000

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((server_ip, port))
print(f"Connected to server {server_ip}:{port}")

while True:
    message = input("Client: ")
    client_socket.send(message.encode())
    if message.lower() == "exit":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)
    if reply.lower() == "exit":
        break

client_socket.close()
```

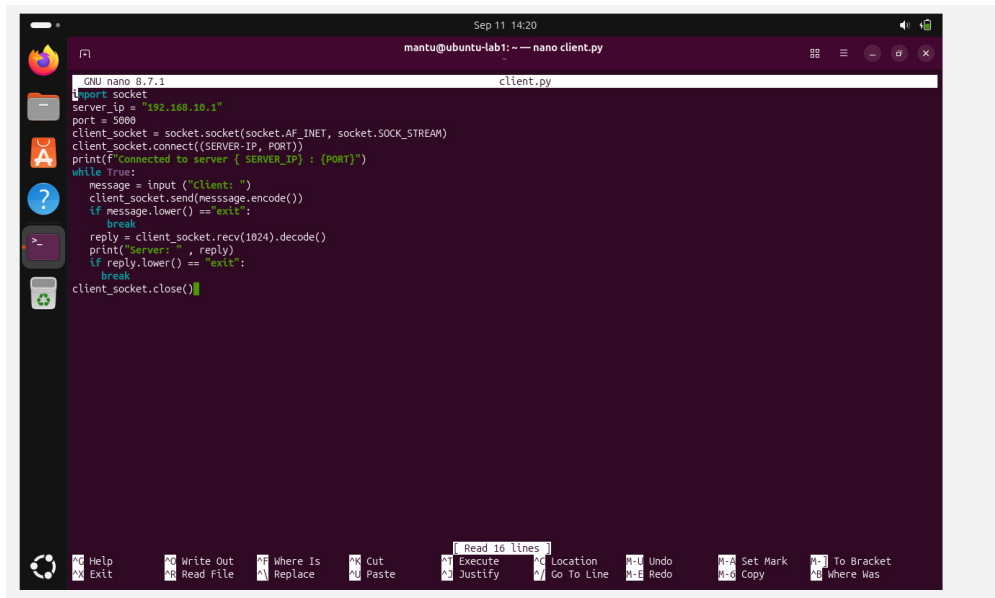


Fig. 5 — `client.py` open in the nano editor on the Client VM.

Running the Programs

The server was launched first on the Server VM using **python3 server.py**, which began listening on 192.168.10.1:5000. The client was then started on the Client VM with **python3 client.py**, which connected to the waiting server. From there, messages were typed back and forth on both terminals, showing the two-way TCP chat working as intended.

```
mantu@ubuntu-lab1:~$ nano server.py
mantu@ubuntu-lab1:~$ python3 server.py
Server started on 192.168.10.1 : 5000
Waiting for client...
Connected by ('192.168.10.2', 46480)
Client disconnectd.
mantu@ubuntu-lab1:~$ python3 server.py
Server started on 192.168.10.1 : 5000
Waiting for client...
Connected by ('192.168.10.2', 58202)
Client: Hello server
server: Hello client
Client: How are you?
server: I am fine
```

Fig. 6 — Server terminal showing the started server, the accepted connection, and the exchanged messages.

```
mantu@ubuntu-lab1:~$ nano client.py
mantu@ubuntu-lab1:~$ python3 client.py
Connected to server 192.168.10.1 : 5000
Client: Hello server
Server: Hello client
Client: How are you?
Server: I am fine
Client: █
```

Fig. 7 — Client terminal showing the connection to the server and the chat in progress.

Observations

- Both virtual machines were placed on one virtual network and given fixed IPs — 192.168.10.1 for the Server and 192.168.10.2 for the Client.
- Reachability between the two VMs was checked and confirmed before the chat program was run.
- The server accepted the client's connection request, after which messages flowed correctly in both directions.
- Either machine could end the chat cleanly by sending the word "exit".

Conclusion

This experiment successfully built and demonstrated a simple TCP client–server chat program running across two separate Ubuntu virtual machines in VirtualBox. It verified static IP configuration, confirmed connectivity between the VMs, and showed reliable two-way messaging using Python sockets — reinforcing the core ideas behind socket programming and TCP-based communication.